The Ultimate Git and GitHub Beginner Cheatsheet

Git is your ultimate safety net as a developer, allowing you to track changes and collaborate without fear of breaking things. Mastering these fundamental commands will save you hours of frustration and boost your confidence daily.

What you'll learn:

1. How to Save Your Work
2. Creating a Safe Sandbox
3. Undoing Local Mistakes
4. Sending Code to the Cloud

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

How to Save Your Work

Before Git saves your work, you have to do two things. First, you stage your files, which is like putting items into a shipping box. Second, you commit them, which is like taping the box shut and writing a label on it. This creates a permanent, searchable snapshot of your code.

```
$ git add index.html
git commit -m "fix: resolved navigation bar alignment issue"
```

2

TIP 2 OF 4

Creating a Safe Sandbox

You should never write new code directly on the main branch where the working app lives. Instead, create a branch, which acts as a safe parallel universe. You can write, break, and test your code here without affecting the live application. Once it is perfect, you can merge it back.

```
$ git checkout -b feature/user-profile
```

3

TIP 3 OF 4

Undoing Local Mistakes

We all make mistakes and sometimes write code that completely breaks our app. Git makes it incredibly easy to discard your unstaged changes and start fresh. Running the restore command will throw away your recent edits to a specific file and bring back the last saved version instantly.

```
$ git restore config.py
```

4

TIP 4 OF 4

Sending Code to the Cloud

After saving your work locally on your computer, you need to send it to GitHub so your team can see it. Pushing uploads your commits from your local repository to the remote repository in the cloud. This serves as an off-site backup and allows others to collaborate on your code.

```
$git push origin feature/user-profile
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- Write clear, action-oriented commit messages (e.g., 'fix: update api' instead of 'stuff').
- Always run 'git status' before committing to see exactly what you are saving.
- Create a '.gitignore' file on day one to keep node_modules and API keys out of your repo.
- Pull the latest changes from your team before you start writing any new code.
- Keep your branches small and focused on one specific feature or bug fix.

Watch Out For

- Committing directly to the main or master branch instead of using feature branches.
- Writing massive commits that contain five different unrelated changes at once.
- Forgetting to configure your Git user name and email before making your first commit.
- Accidentally pushing sensitive credentials, passwords, or .env files to public repositories.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh